In God we trust

نام و نام خانوادگی : مهدیس میرزایی

شماره دانشجویی : ۸۱۰۱۰۲۵۵۰

شماره پروژه : پروژه دوم

بخش یک :

۱ - با استفاده از تابع uigetfile خواسته سوال که باز شدن پنجره‌ای برای دریافت عکس پلاک ماشین است را انجام می‌دهیم و با استفاده از fullfile هر دو اسم فایل و آدرس آن را یکی می‌کنیم.

```
clc; clear; close all;
[filename, pathname] = uigetfile({'*.jpg;*.png;*.bmp','Image Files'}, 'Select image');
img = imread(fullfile(pathname, filename));
```

پس داریم :



۲- با استفاده از imresize آن را به ابعاد ۳۰۰ در ۵۰۰ تبدیل می‌کنیم و آن را چاپ می‌کنیم.

```
img_resized = imresize(img, [300 500]);
figure, imshow(img_resized);
title('Resized Image');
```

پس داریم :



۳- طبق صورت پروژه جهت کاهش پیچیدگی‌های محاسباتی رنگ‌ها را حذف می‌کنیم و مانند رابطه گفته شده پیش می‌رویم پس تابعی برای آن ایجاد می‌کنیم و داریم :

```
function gray_img = mygrayfun(rgb_img)
R = double(rgb_img(:,:,1));
G = double(rgb_img(:,:,2));
B = double(rgb_img(:,:,3));
gray_img = uint8(0.299*R + 0.578*G + 0.114*B);
end
```

و با اعمال آن در کد برای عکس قبلی خواهیم داشت :

```
gray_img = mygrayfun(img_resized);
figure, imshow(gray_img);
title('Grayscale Image');
```

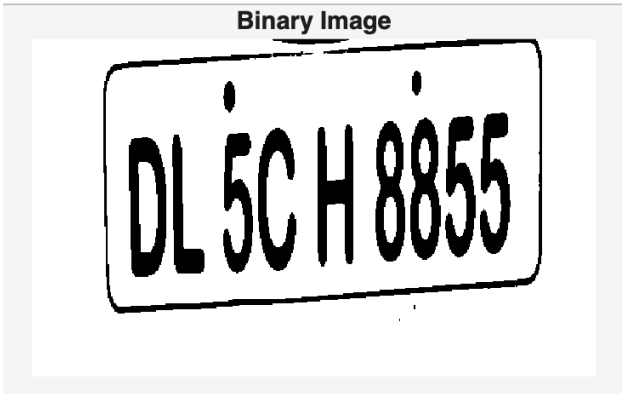


۴- حال برای تبدیل عکس به باینری - تابعی ایجاد می‌کنیم و داریم :

```
function bin_img = mybinaryfun(gray_img, threshold)
bin_img = gray_img > threshold;
end
```

و با اعمال آن در کد برای عکس قبلی خواهیم داشت :

```
threshold = 127;
bin_img = ~mybinaryfun(gray_img, threshold);
figure, imshow(bin_img);
title('Binary Image');
```

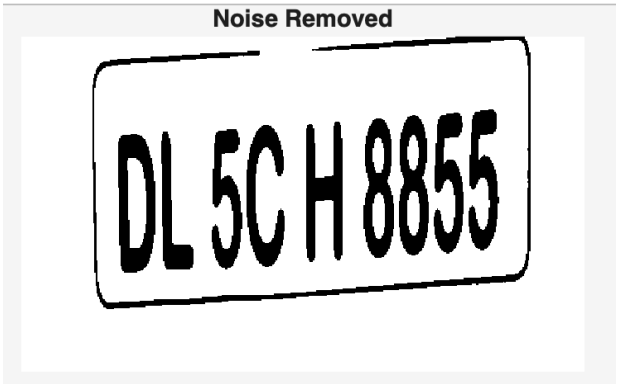


۵- در این قسمت در واقع داریم به ازای هر یک از خانه‌های جدول یکبار الگوریتم bfs را اجرا می‌کنیم و اگر سایز مولفه‌ای که در آن قرار گرفت از n بیشتر باشد آن و مولفه‌اش را علامت می‌زنیم. به عبارتی می‌توان گفت این‌کار را انجام می‌دهیم تا اگر سایز مولفه‌های همبندی را به دست آوریم و سپس اگر این مقدار کم بود آن را مانند نویز تلقی کنیم و حذف کنم.

```
function cleanpic = myremovecom(pic, thresh)
    [rows, cols] = size(pic);
    A = pic;
    segpic = zeros(rows, cols);
    N = 0;
    dirs = [ -1 0; 1 0; 0 -1; 0 1; -1 -1; -1 1; 1 -1; 1 1 ];
    for i = 1:rows
        for j = 1:cols
            if A(i,j) == 1
                N = N + 1;
                stack = [i, j];
                while ~isempty(stack)
                    r = stack(end,1);
                    c = stack(end,2);
                    stack(end,:) = [];
                    if r<1 || r>rows || c<1 || c>cols
                        continue;
                    end
                    if A(r,c) == 0
                        continue;
                    end
                    A(r,c) = 0;
                    segpic(r,c) = N;
                    for k = 1:8
                        rr = r + dirs(k,1);
                        cc = c + dirs(k,2);
                        if rr>=1 && rr<=rows && cc>=1 && cc<=cols
                            if A(rr,cc)==1
                                stack = [stack; rr, cc];
                            end
                        end
                    end
                end
            end
        end
    end
    componentAreas = zeros(1, N);
    for label = 1:N
        componentAreas(label) = sum(segpic(:) == label);
    end
    validLabels = find(componentAreas >= thresh);
    cleanpic = ismember(segpic, validLabels);
end
```

و پس از اعمال آن روی کد داریم :

```
n = 250;
clean_img = myremovecom(bin_img, n);
figure, imshow(clean_img);
title('Noise Removed');
```



۶- در این قسمت می‌خواهیم مانند قسمت الگوریتمی شبیه bfs انجام دهیم اما با هدف اینکه هر یک از نواحی را به طور جدا برچسب‌گذاری کنیم. از تابع imdilate استفاده کرده‌ایم. این تابع به این صورت است که هربار از تمامی ۸ جهت بررسی می‌کند آیا این ناحیه را می‌توان افزایش دهد به عنوان یک mask جدید خروجی تابع را با خانه ابتدایی AND میکند تا آن خانه باقی بماند. از طرفی اگر تغییری ایجاد شده باشد فرایند ادامه پیدا می‌کند و در غیر این صورت یعنی آن ناحیه را دیگر نمی‌توان افزایش داد.

```
function [segpic , N] = mysegmentation(pic)
pic = logical(pic);
A = pic;
segpic = zeros(size(A));
N = 0;
se = strel('square',3);
while any(A(:))
    [r,c] = find(A,1);
    mask = false(size(A));
    mask(r,c) = true;
    done = false;
    while ~done
        oldmask = mask;
        mask = imdilate(mask,se) & A;
        done = isequal(mask,oldmask);
    end
    N = N + 1;
    segpic(mask) = N;
    A(mask) = 0;
end
end
```

و آن را در کد اعمال می‌کنیم.

```
[L, num] = mysegmentation(clean_img);
figure, imshow(L);
title('Segmented Image');
```



۷- در این قسمت می‌خواهیم مپ ستی که داریم را لود کنیم و سپس به ازای سگمنت‌هایی که به دست آورده‌ایم با هریک از داده‌های مپ‌ست correlation محاسبه کنیم و البته مقداری به عنوان Threshold قرار می‌دهیم که اگر داده نویزی هنوز موجود باشد (مثل کادر) حذف شود. برای پاک کردن کادر البته از روش سر کلاس که به تابع نویز با n ای بزرگ می‌دهیم و این باعث می‌شود که با کم کردن مقدار قبلی و مقدار بدون اعداد - صرفا اعداد و حروف برای ما باقی بماند. در ابتدا با توجه مپ‌ستی که داده شده است و label هایی که در آن موجود است دیتاها را مرتب می‌کنیم.

```
files=dir('Map Set\');
len = length(files) - 2;
train = cell(2, len);
for i = 1 : len
   img = imread(fullfile('Map Set', files(i + 2).name));
   label =files(i+2).name(1);
   train{1, i} = img;
   train{2, i} = label;
end
save('train.mat','train');
load train;
all_letters=size(train,2);
figure
final_result=[];
for n = 1 : num
    [r, c] = find(L == n);
    Y = L(min(r) : max(r), min(c) : max(c));
    imshow(Y)
    Y=imresize(Y, [42,24]);
    imshow(Y)
    pause(0.2)
    ro=zeros(1, all_letters);
    for k = 1 : all_letters
        ro(k) = corr2(train{1,k}, Y);
    end
    [MAXRO,pos] = max(ro);
    if MAXRO>.47
        out=cell2mat(train(2,pos));
        final_result=[final_result out];
    end
end
```

۸- با لود کردن دیتا ها و محاسبات corr در final result نتیجه نهایی را داریم و آن را در فایل answer ذخیره می‌کنیم.

```
disp(final_result);
type('answer.txt');
```

و خروجی answer را داریم :



بخش دو :

در این بخش مانند بخش اول عمل می کنیم و صرفا با دیتا های فارسی ران می کنیم.

nanih\OneDrive\Desktop\mahids\p2\p2.m

```
clc; clear; close all;
[filename, pathname] = uigetfile({'*.jpg;*.png;*.bmp','Image Files'}, 'Select image');
img = imread(fullfile(pathname, filename));
img_resized = imresize(img, [300 500]);
figure, imshow(img_resized);
title('Resized Image');
gray_img = rgb2gray(img_resized);
figure, imshow(gray_img);
title('Grayscale Image');
bin_img = ~imbinarize(gray_img, graythresh(gray_img));
figure, imshow(bin_img);
title('Binary Image');
n = 500;
background = bwareaopen(bin_img, 4000);
clean_img = bwareaopen(bin_img, n);
clean_img = clean_img & ~background;
figure, imshow(clean_img);
title('Noise Removed');
[L, num] = bwlabel(clean_img);
figure, imshow(label2rgb(L));
title('Segmented Image');
files = dir('Map Set\');
len = length(files) - 2;
train = cell(2, len);
for i = 1 : len
   img = imread(fullfile('Map Set', files(i + 2).name));
   if ndims(img) == 3
       img = rgb2gray(img);
   end
   label = files(i + 2).name(1);
   train{1, i} = img;
   train{2, i} = label;
end
all_letters = size(train, 2);
figure
final_result = [];
for n = 1 : num
    [r, c] = find(L == n);
    Y = L(min(r) : max(r), min(c) : max(c));
    imshow(Y)
    Y = imresize(Y, [42, 24]);
    imshow(Y)
    pause(0.2)
```

و عکس پلاک و خروجی کد را در ادامه داریم :

```
39        Y = L(min(r) : max(r), min(c) : max(c));
40        imshow(Y)
```

Command Window

```
27M13322

27M13322
>>
```

و پلاک را داریم :



بخش سوم :

برای این قسمت بخش آبی تصویر را از ورژن RGB عکس را نگه میداریم. به عبارتی تمام عکس را سیاه میکنیم و قسمت آبی را سفید میکنیم. مانند قسمت الف نویزها را حذف و سپس segmentation انجام می‌دهیم. این باعث می‌شود که بتوان با توجه به آن تمامی پلاک را تشخیص داد. چون اندازه مستطیل حدودا ۱۲ برابر کادر آبی است پس می‌توان با توجه به این اسکیل پلاک را به طور کامل تشخیص داد.

```
[file, path] = uigetfile({'*.jpg;*.bmp;*.png;*.tif'}, 'Select image');
imagePath = fullfile(path, file);
carImg = imread(imagePath);
figure; imshow(carImg); title('Car');
targetSize = [500 800];
carImg = imresize(carImg, targetSize);
figure; imshow(carImg); title('Resized Image');
redCh   = carImg(:,:,1);
greenCh = carImg(:,:,2);
blueCh  = carImg(:,:,3);
blueMask = (blueCh > 100) & (blueCh > 1.25 * redCh) & (blueCh > 1.15 * greenCh);
figure; imshow(blueMask); title('Blue');
minBlobSize = 60;
blueMask = bwareaopen(blueMask, minBlobSize);
figure; imshow(blueMask); title('Filtered Blue');
[labeledMask, regionCount] = bwlabel(blueMask);
regions = regionprops(labeledMask, 'BoundingBox', 'Area');
potentialIdx = [];
for i = 1:regionCount
    bbox = regions(i).BoundingBox;
    width  = bbox(3);
    height = bbox(4);
    aspectRatio = height / width;
    if aspectRatio > 1.6 && aspectRatio < 6 && ...
       width > 10 && height > 25 && ...
       bbox(1) < size(carImg,2)/1.8
        potentialIdx = [potentialIdx i];
    end
end
[~, chosenIdx] = max([regions(potentialIdx).Area]);
stripBox = regions(potentialIdx(chosenIdx)).BoundingBox;
```

و عکس پلاک ماشین و پلاک تشخیص داده شده را در ادامه داریم :

```
>> p3
59Y84410
```



بخش چهارم :

در این قسمت از ویدیو برای هر فریم، تفاوت آن با فریم قبلی را نگه می‌داریم. به عبارتی قسمت‌هایی از تصویر که تغییر کرده‌اند (نواحی متحرک) سفید و بقیه بخش‌ها سیاه می‌شوند. سپس مانند مراحل قبل نویزها حذف و نواحی پیوسته (segmentation) انجام می‌شود. با انتخاب بزرگ‌ترین ناحیه سفید، جسم اصلی در حال حرکت تشخیص داده می‌شود. با دنبال‌کردن مرکز این ناحیه در فریم‌های متوالی، مسیر حرکت جسم و در نهایت سرعت آن به دست می‌آید.

```
clc; clear; close all;
vidName = 'video.mp4';
vObj = VideoReader(vidName);
fps = vObj.FrameRate;
timeStep = 1 / fps;
fprintf('Loaded "%s" [%.2f frames/sec]\n', vidName, fps);
firstFrame = readFrame(vObj);
grayPrev = rgb2gray(firstFrame);
motionCenters = [];
frameIndex = 1;
storedFrames = {};
maxShow = 14;
colsPerRow = 7;
while hasFrame(vObj)
    frameRGB = readFrame(vObj);
    grayCurr = rgb2gray(frameRGB);
    diffImg = imabsdiff(grayCurr, grayPrev);
    motionMask = diffImg > 50;
    motionMask = bwareaopen(motionMask, 180);
    motionMask = imfill(motionMask, 'holes');
    regions = regionprops(motionMask, 'Centroid', 'Area', 'BoundingBox');
    fTemp = figure('Visible','off');
    imshow(grayCurr); hold on;
    if ~isempty(regions)
        [~, biggestIdx] = max([regions.Area]);
        bbox = regions(biggestIdx).BoundingBox;
        centerPt = regions(biggestIdx).Centroid;
        motionCenters = [motionCenters; centerPt];
        rectangle('Position', bbox, 'EdgeColor', 'cyan', 'LineWidth', 2);
        plot(centerPt(1), centerPt(2), 'y+', 'MarkerSize', 10, 'LineWidth', 1.5);
    end
    title(sprintf('Frame #%d', frameIndex));
    frameGrab = getframe(gca);
    storedFrames{end+1} = frameGrab.cdata;
    close(fTemp);
    grayPrev = grayCurr;
    frameIndex = frameIndex + 1;
end
fprintf('Total processed frames: %d\n', frameIndex);
if size(motionCenters,1) < 2
    error('Not enough tracked points for motion estimation.');
end
verticalShift = diff(motionCenters(:,2));
pixelVelocity = verticalShift / timeStep;
averageVelocity = mean(pixelVelocity, 'omitnan');
fprintf('Average pixel velocity: %.2f pixels/sec\n', averageVelocity);
nShow = min(maxShow, numel(storedFrames));
figure;
for i = 1:nShow
    subplot(ceil(nShow/colsPerRow), colsPerRow, i);
    imshow(storedFrames{i});
    title(sprintf('Frame %d', i));
end
sgtitle('Motion Tracking Sequence');
```

Command Window

```
Loaded "video.mp4" [30.00 frames/sec]
Total processed frames: 98
Average pixel velocity: -25.20 pixels/sec
>>
```

